LIFELINE: System Architecture Proposal & Team Contributions

1. Team Member Contributions & Module Assignments

Member	Contribution (Iteration 1 Deliverables & Proposed Iteration 2 Modules)
Ch Faizan Zahid Roll No: 221-0533 Section: B	**Iteration 1:** Generator architecture, knapsack/claims exact solver (crates, claims sheets), graph generator & closures (roads, waves sheets), validator core engine (V1-V5 checks), approach note part 1 & 2. **Proposed Iteration 2 Modules:** Module A (Relief Inventory), Module B (Route Planning), Module H (Hard Planning Questions: 3-SAT & TSP).
Mohid Abbas Roll No: 221-0074 Section: B	**Iteration 1:** Incident & rescue window generators (incidents, windows sheets), radio log & report generators (radio_log, radio_reports sheets), validator checks (V6-V7 signature & variety checks), validator test suite (broken test cases), approach note part 3 & 4. **Proposed Iteration 2 Modules:** Module C (Rescue Scheduling), Module D (Emergency Queue), Module E (Radio Logs).
Fizan Tariq Roll No: 221-0547	**Iteration 1:** Geometric sites & sensor network generators (sites, tour, sensor_edges, sensor_cover sheets),

Member	Contribution (Iteration 1 Deliverables & Proposed Iteration 2 Modules)
Section: B	pipeline & drum generators (pipeline_dims, drum_prices, rations sheets), validator trap checks (V8 checks), validation report compilation, repository & packaging. **Proposed Iteration 2 Modules:** Module F (Network Planning), Module G (Chains and Cuts), Module I (Nearest Sites and Checksums).

2. Algorithmic Modules & Technical Specifications

Module	Problem (Variables: n, m, V, E, W)	Candidates (with Complexity)	Selection (Primary vs Baseline)	Justification (Data Property, Assumptions & Failure Case)
A. Relief inventory	0/1 Knapsack. Input: n crates with weights w_i in [1, 20] kg, values v_i in [10, 100], truck capacity W ≤ ∑w_i. Output: Set S ⊆ {1..n} maximizing total value ∑v_i	1. 1D Tabular DP: O(n · W) pseudo-poly. 2. Branch and Bound: O(2^n) worst-case. 3. Greedy by v/w ratio: O(n log n).	**Primary:** 1D Dynamic Programming O(n · W). **Baseline:** Greedy value-density-heuristic O(n log n).	Weights and capacity are strictly positive integers with W ≤ 30,000, where pseudo-polynomial DP runs in milliseconds. Assumes static integer crate weights. Fails if

Module	Problem (Variables: n, m, V, E, W)	Candidates (with Complexity)	Selection (Primary vs Baseline)	Justification (Data Property, Assumptions & Failure Case)
	subject to total weight ∑w_i ≤ W.			weights/capacities are continuous non-integral reals or capacity W >> 10^7.
B. Route planning	Single-Source Shortest Paths & Reachability. Input: Directed graph G=(V,E) with road traversal time minutes(e) in [1, 60] and destination penalty(v) in {0,1,2,4,6}. Output: Cheapest path minimizing ∑(minutes + penalty_v), reachable set, and min-hop path.	1. Dijkstra with Min-Heap: O((V+E) log V). 2. Bellman-Ford: O(V · E). 3. Unweighted BFS: O(V+E) for hop count.	**Primary:** Dijkstra with Min-Heap O((V+E) log V) and BFS O(V+E). **Baseline:** Bellman-Ford O(V · E).	Minutes and penalties are non-negative (edge cost ≥ 1), perfectly satisfying Dijkstra's greedy non-negative edge invariant. Assumes static road state per day. Fails if negative cost edges or negative penalty-relief cycles exist.

Module	Problem (Variables: n, m, V, E, W)	Candidates (with Complexity)	Selection (Primary vs Baseline)	Justification (Data Property, Assumptions & Failure Case)
C. Rescue scheduling	Interval Scheduling & Weighted Interval Scheduling. Input: m rescue windows [start_i, finish_i] with values v_i. Output: Maximum cardinality compatible set (touching allowed) or maximum value compatible set.	1. Greedy Earliest Finish Time (EFT): O(m log m). 2. Weighted Interval DP with Bisect: O(m log m). 3. Integer Linear Program: $O(2^m)$.	**Primary:** Greedy EFT for count; DP + Binary Search for value O(m log m). **Baseline:** Recursive search $O(2^m)$.	Intervals are 1D continuous segments [0, 1440]. Sorted finish times guarantee the greedy-choice property for count and optimal substructure for weighted DP via binary search predecessor lookup. Fails if windows require multi-crew matching or have setup dependencies.
D. Emergency queue	Dynamic Priority Queue Simulation. Input: A alarms arriving at non-decreasing tick t_i with	1. Dial's Bucket Queue/Multi-level Array: O(A+K) where K=10. 2. Binary Max-Heap: O(A	**Primary:** Dial's Multi-level Bucket Queue O(A) total, O(1) amortized. **Baseline:**	Priority values are small, discrete integers bounded in [1, 10]. Direct array buckets

Module	Problem (Variables: n, m, V, E, W)	Candidates (with Complexity)	Selection (Primary vs Baseline)	Justification (Data Property, Assumptions & Failure Case)
	priority p_i in [1, 10] (10 most urgent). Output: Release order answering one alarm per tick in urgency order among available alarms.	log A). 3. Unsorted Scan: $O(A^2)$.	Binary Max-Heap O(A log A).	permit O(1) insertion and O(1) extraction of the highest-priority incident. Fails if priorities are arbitrary continuous real numbers or priority space K >> A.
E. Radio logs	Exact Pattern Search, String Edit Distance & Data Compression. Input: Alphabet Σ (|Σ|=38), log length N, reports of lengths L_a, L_b. Output: All phrase occurrences, min edit script (ins/del/sub), and optimal prefix-free	1. Knuth-Morris-Pratt (KMP): O(N+M) search. 2. Wagner-Fischer DP: O(L_a · L_b) edit dist. 3. Huffman Greedy Tree: O(|Σ| log |Σ|).	**Primary:** KMP search O(N+M), Wagner-Fischer DP O(L_a · L_b), Huffman O(|Σ| log |Σ|). **Baseline:** Naive search O(N · M).	Fixed discrete alphabet (|Σ|=38) and static log text allow linear deterministic automata / failure function preprocessing. Character frequencies in logs exhibit strong skewness (e.g. Study 6 and Trap 49), guaranteeing

Module	Problem (Variables: n, m, V, E, W)	Candidates (with Complexity)	Selection (Primary vs Baseline)	Justification (Data Property, Assumptions & Failure Case)
	code.			Huffman optimality. Fails under wildcard/regex stream queries.
F. Network planning	Minimum Spanning Tree (MST). Input: Connected reachable graph G=(V,E) with cable costs c_e in [1, 100]. Output: Minimum cable cost acyclic subgraph connecting all reachable locations.	1. Kruskal with DSU & Counting Sort: $O(E \cdot \alpha(V))$. 2. Prim with Binary Heap: $O(E \log V)$. 3. Boruvka's Algorithm: $O(E \log V)$.	**Primary:** Kruskal with Disjoint Set Union (DSU) $O(E \cdot \alpha(V))$. **Baseline:** Prim's Algorithm $O(E \log V)$.	Road network is relatively sparse ($E \leq 3V$) and cable costs are integers in [1, 100]. Edge weights can be sorted in linear $O(E)$ time using Counting Sort, reducing Kruskal to nearly $O(E)$. Fails on dense graphs where $E = \Theta(V^2)$ and Prim with dense matrix is $O(V^2)$.
G. Chains and cuts	Matrix Chain Multiplication (MCM) & Rod	1. Bottom-up Dynamic Programming:	**Primary:** Tabular Dynamic	Both problems possess strict optimal

Module	Problem (Variables: n, m, V, E, W)	Candidates (with Complexity)	Selection (Primary vs Baseline)	Justification (Data Property, Assumptions & Failure Case)
	Cutting. Input: n pipeline stage dimensions; drum length L with price array P[1..L]. Output: Optimal associative parenthesization minimizing scalar multiplications; optimal cutting maximizing total revenue.	$O(n^3)$ MCM, $O(L^2)$ Rod Cut. 2. Memoized Recursion: $O(n^3)$ MCM, $O(L^2)$ Rod Cut. 3. Greedy Heuristic: O(n log n) MCM.	Programming $O(n^3)$ MCM, $O(L^2)$ Rod Cut. **Baseline:** Greedy dimensional/density heuristic.	substructure and overlapping subproblems without cyclic dependencies. Stage counts (n ≤ 300) and drum lengths (L ≤ 2000) evaluate rapidly under tabular DP. Fails if drum cutting incurs cutting kerf material loss or stage operators are non-associative.
H. Hard planning questions	NP-Hard Combinatorial Decision Problems: 3-SAT (Safety), Exact Subset Sum (Rations), Metric TSP (Tour), and	1. CDCL/DPLL with Unit Propagation for 3-SAT. 2. Meet-in-the-Middle / DP: $O(2^{n/2})$ or O(n · Target).	**Primary:** CDCL for 3-SAT, Meet-in-the-Middle for Rations, Held-Karp (Tier S) / 2-Opt (L/X) for TSP, FPT B&B for MVC.	Planted solutions exist by construction (planted cover k ≤ n/2, planted assignment, planted subset). Small parameter

Module	Problem (Variables: n, m, V, E, W)	Candidates (with Complexity)	Selection (Primary vs Baseline)	Justification (Data Property, Assumptions & Failure Case)
	Minimum Vertex Cover (Sensors). Output: Exact satisfying assignment, exact target subset, shortest tour, and min cover.	3. Held-Karp DP $O(n^2 \cdot 2^n)$ / 2-Opt. 4. FPT Branch & Bound: $O(1.2738^k + kn)$.	**Baseline:** Brute-force $O(2^n)$ / $O(n!)$.	sizes on Tier S ($n \leq 15$) allow exact Held-Karp/FPT algorithms to verify claims. Fails on unconstrained instances beyond the 3-SAT phase transition ratio (rules/vars 4.267).
I. Nearest sites and checksums	2D Closest Pair of Points & Arbitrary-Precision Multiplication. Input: n coordinates (x, y) in $[0, 999]^2$; two long decimal numbers from concatenated crate serials (d digits). Output: Minimum	1. Divide and Conquer Closest Pair: $O(n \log n)$. 2. Karatsuba Multiplication: $O(d^{1.585})$. 3. Naive Pairwise & Schoolbook: $O(n^2)$ and $O(d^2)$.	**Primary:** Divide-and-Conquer Closest Pair $O(n \log n)$ and Karatsuba $O(d^{1.585})$. **Baseline:** Pairwise distance $O(n^2)$ and Schoolbook multiplication $O(d^2)$.	Sites lie in a bounded Euclidean plane where recursive geometric strip pruning guarantees at most 7 candidates per point. Serial digit lengths d reach thousands of digits in Tier L

Module	Problem (Variables: n, m, V, E, W)	Candidates (with Complexity)	Selection (Primary vs Baseline)	Justification (Data Property, Assumptions & Failure Case)
	squared Euclidean distance; high-precision product.			where Karatsuba achieves substantial sub-quadratic speedup. Fails for points in high dimensions (d >> 10).